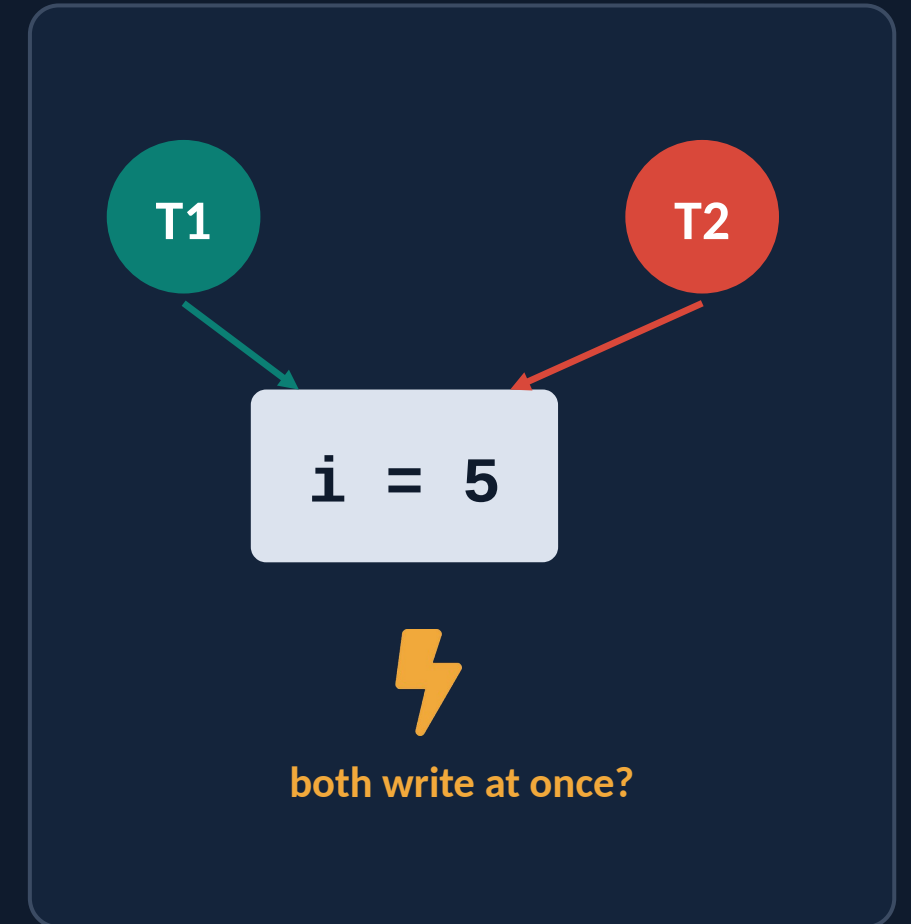


Pthreads & Synchronization

Why shared memory is powerful, why it is dangerous, and how to make it safe

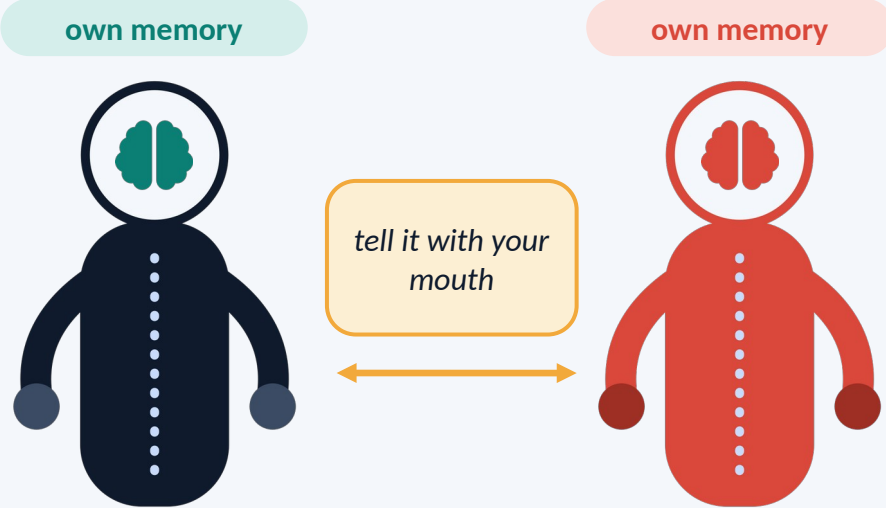
Builds on `fork()` · `pthread_create()` · `pthread_join()`



Clone or extra hand?

RECAP

fork() = a clone of the person

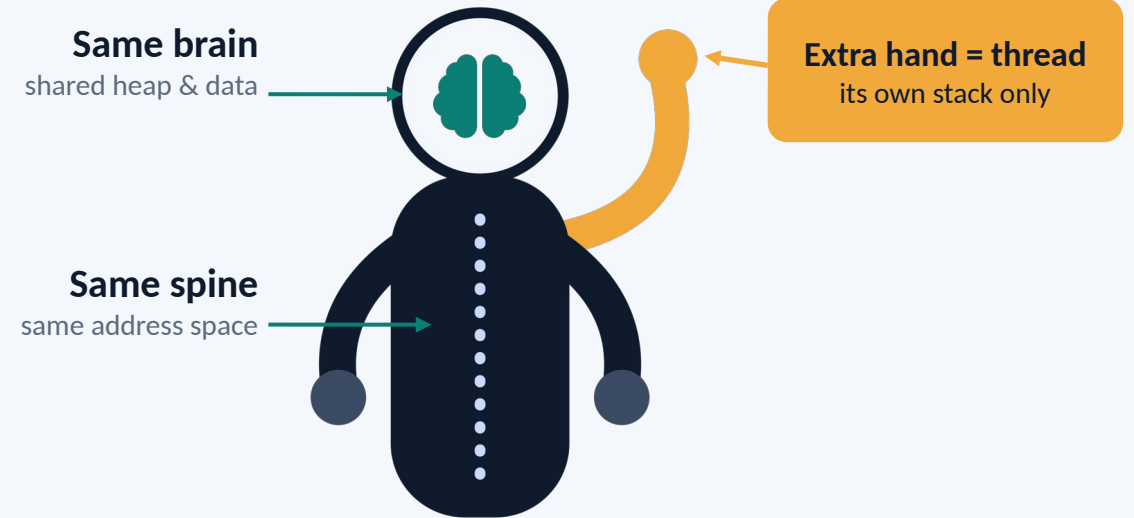


New memory space

Independent processes

Only messages between them

thread = an extra hand



Shared memory space

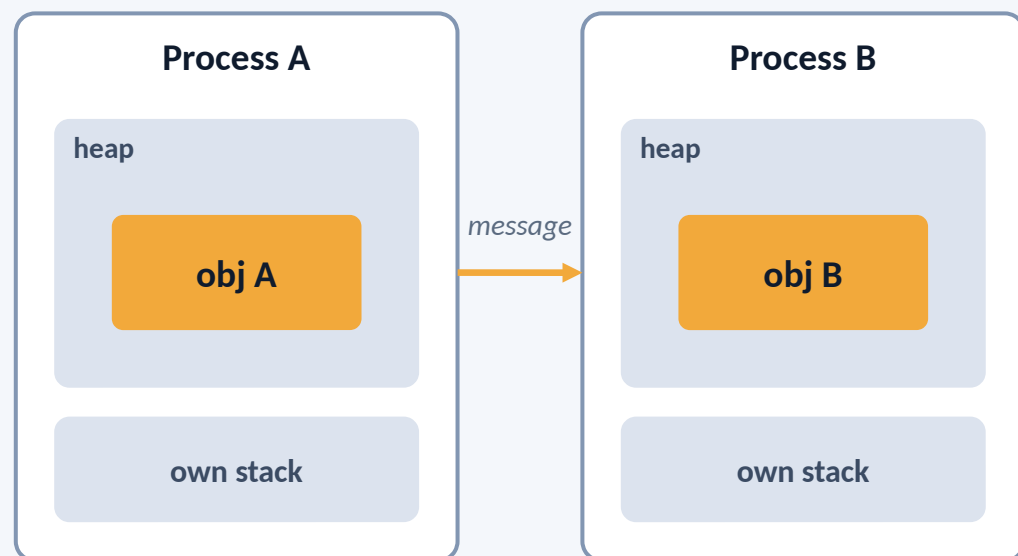
Only its own stack is separate

Talk through shared variables

Small task? Use threads.

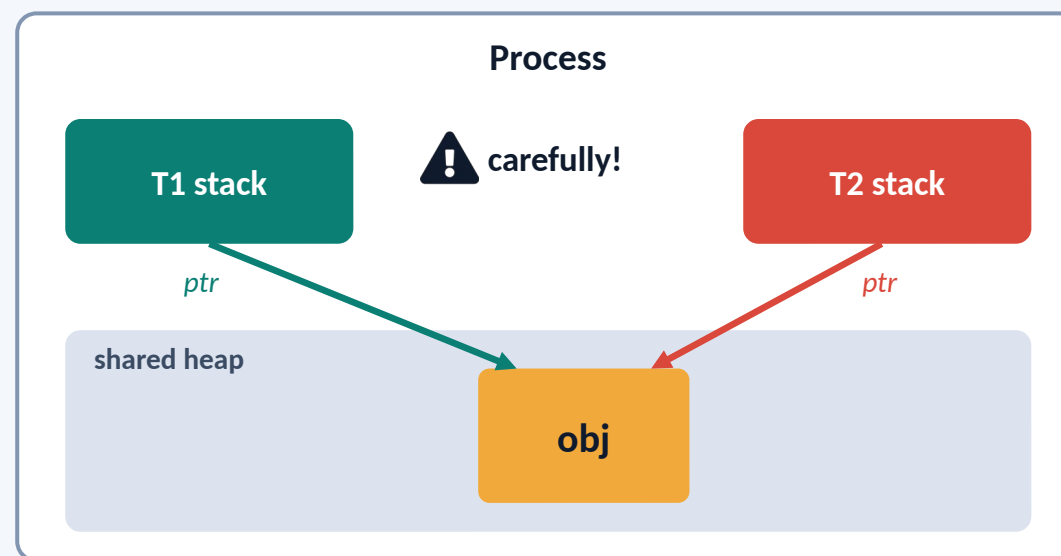
WHY THREADS

fork() = separate heaps



Costly: clone everything, and no shared pointers, only messages.

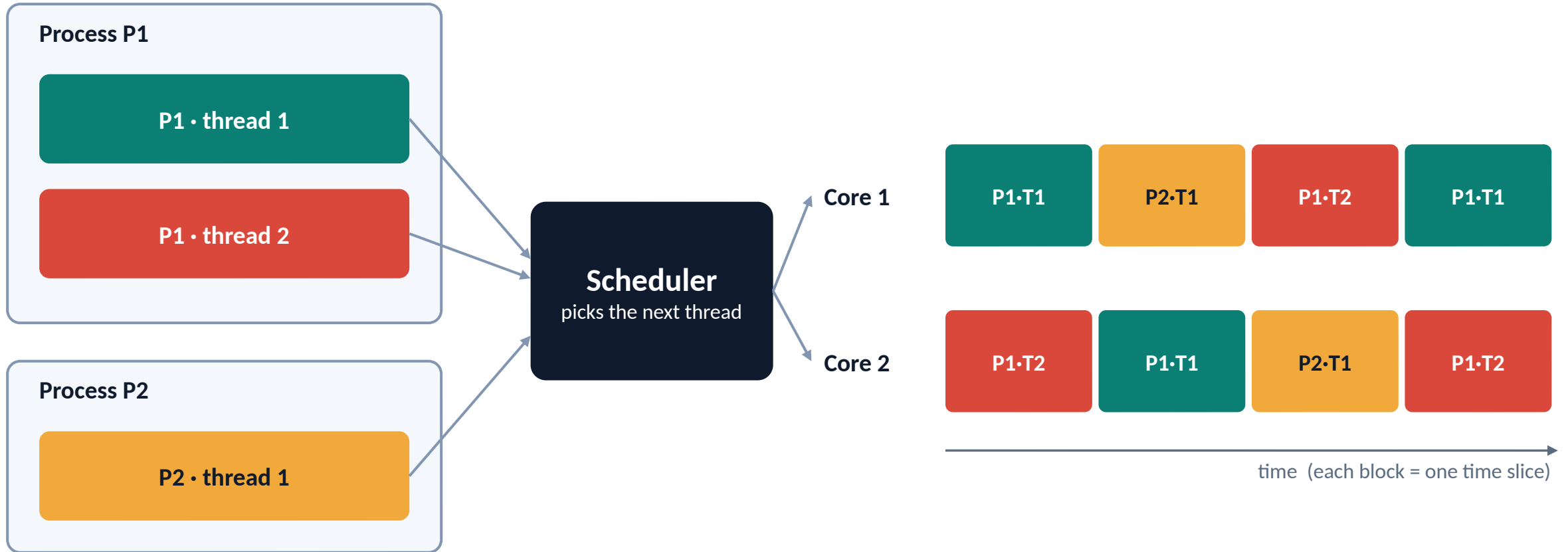
threads = one shared heap



Cheap: same heap, just a new stack. Objects and pointers work from many threads.

What does the scheduler schedule?

RECAP

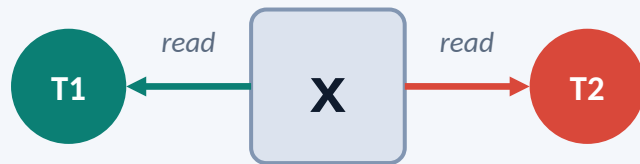


Scheduling is about threads, not processes. A process with a single thread is just the simplest case.

When does sharing become a problem?

THE PROBLEM

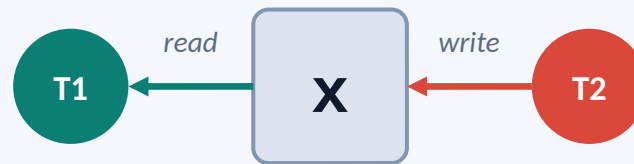
Read + Read



Safe

Nobody changes x, so both see the same value.

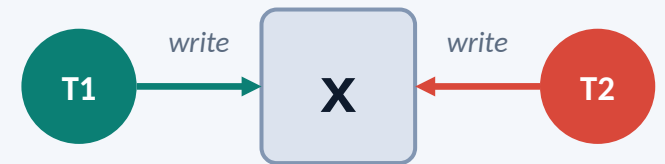
Read + Write



Risky

The reader may see an old or half-updated value.

Write + Write



Broken

The two updates can overwrite each other.

Is i++ really one step?

THE PROBLEM

```
int i = 5;
```

```
void work() {
```

```
...
```

```
i++;
```

```
...
```

```
}
```

Zoom in: what the CPU really runs

1

LOAD

```
mov eax, [i]
```

Get i's value from memory into a register

2

ADD

```
add eax, 1
```

Increment the value in the register

3

STORE

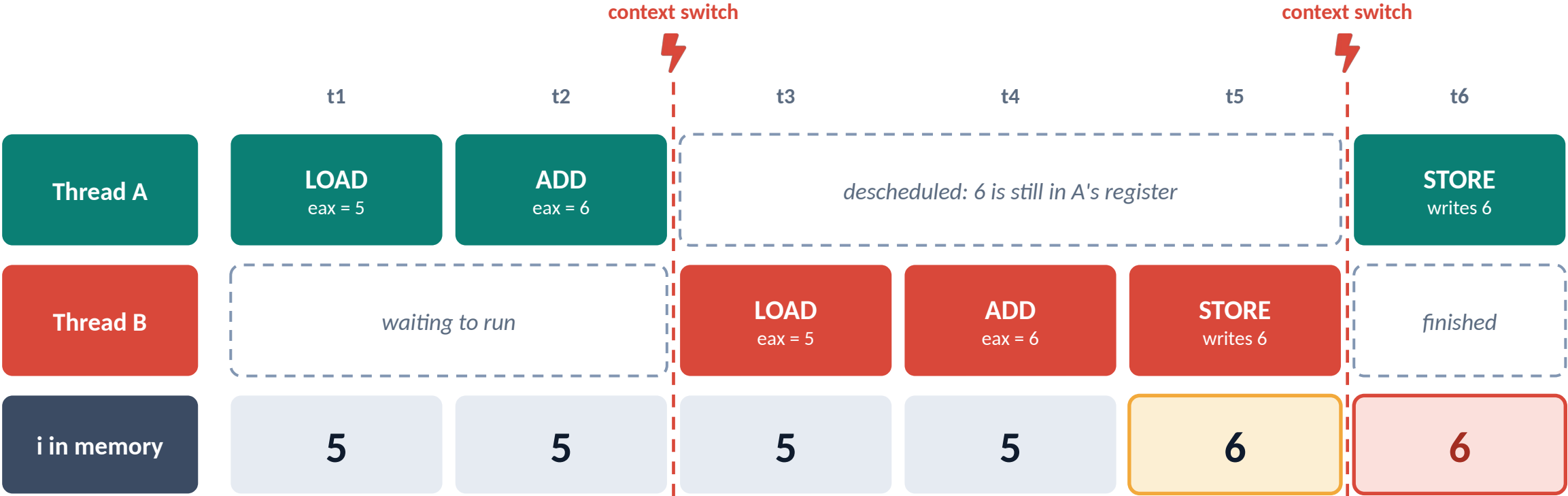
```
mov [i], eax
```

Write the updated value to i's address

3 instructions: i++ is NOT a single, uninterruptible step

What if a context switch hits mid-i++?

THE PROBLEM



A is switched out right before STORE. B then reads the old 5 and stores 6. Later A resumes and stores its stale 6.

Two `i++` ran, but `i` went `5 → 6`
expected 7: one increment was lost

Fix, idea 1: a critical section

IDEA 1



1

lock

enter the section

2

work

use the shared data

3

unlock

let the next one in

Critical section in code

IDEA 1

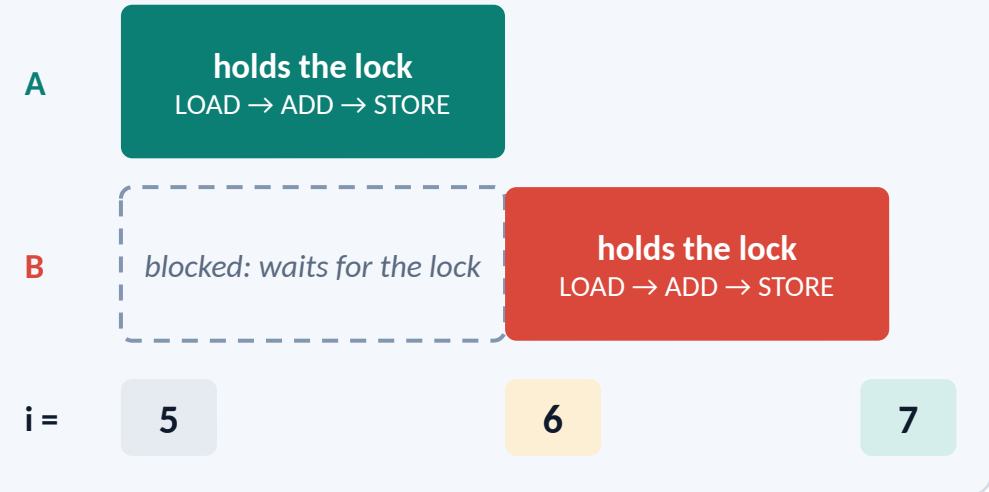
```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int i = 5;

void *work(void *arg) {
    pthread_mutex_lock(&lock);    // enter
    i++;                          // critical section
    pthread_mutex_unlock(&lock);  // leave
    return NULL;
}
```

One lock per shared resource: every access to *i* takes the same lock

Keep the critical section short: others are waiting

With the lock



Both increments count: *i* = 7

B only started once A had unlocked

Fix, idea 2: atomic instructions

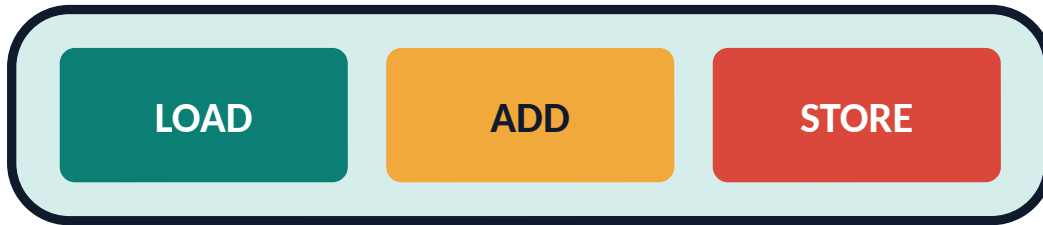
IDEA 2

Plain i++



a context switch can land in these gaps

Atomic i++



one indivisible step: nothing can slip in between

```
#include <stdatomic.h>

atomic_int i = 5;

atomic_fetch_add(&i, 1);
// like i++, but indivisible
```

Mutex (critical section)

Guards any block of code. Other threads wait their turn.

Atomic instruction

One variable, no lock, no waiting. Simple operations only.

Key takeaways

1

Thread = extra hand

Shared memory and heap, only its own stack. `fork()` is a full clone that needs messages.

2

`i++` is three steps

LOAD, ADD, STORE. A context switch between them loses an update.

3

Idea 1: critical section

Lock before touching shared data, unlock after. Only one thread inside at a time.

4

Idea 2: atomic instructions

Make the update one indivisible step. Ideal for a single variable.